

Python Advanced Interview Topics

Framework Development · Scaling · Kafka Deep Dive · Caching
Queue Architectures · Production Stability · AI-First Technologies

Contents

1. Python Framework Development
2. Scaling Applications & Handling Challenges
3. Kafka — Deep Dive Topics
4. Caching Mechanisms
5. Queue-Based Architectures
6. Production Stability — Alerting & Monitoring
7. AI-First Technologies & Solutions

Section A — Python Framework Development

A1. How do you structure a large-scale Python REST API project?

Follow a layered, domain-driven structure. Routers handle HTTP only; services contain business logic; repositories abstract data access. Each layer depends only on the one below it — never skip layers.

```

src/
├─ main.py           # app factory + lifespan + middleware
├─ config.py         # pydantic-settings – env-driven
├─ dependencies.py    # shared FastAPI Depends() providers
├─ routers/
│   ├── __init__.py
│   ├── orders.py     # HTTP layer only – thin
│   └── users.py
├─ services/
│   ├── order_service.py # business rules, orchestration
│   └── user_service.py
├─ repositories/
│   ├── order_repo.py   # DB access – SQLAlchemy / Couchbase
│   └── base_repo.py    # generic CRUD base
├─ models/            # ORM models
├─ schemas/           # Pydantic request/response DTOs
├─ events/            # Kafka producers/consumers
├─ middleware/        # Auth, tracing, rate-limit
└─ tests/
    ├── unit/
    └── integration/

```

The repository pattern is the most important structural decision — it lets you swap PostgreSQL for Couchbase or add a cache layer without touching service code.

A2. How do you build a generic repository base class in Python?

```
from abc import ABC, abstractmethod
from typing import Generic, TypeVar, Optional, List
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select

T = TypeVar("T")

class BaseRepository(ABC, Generic[T]):
    def __init__(self, session: AsyncSession, model):
        self.session = session
        self.model = model

    async def get_by_id(self, id: int) -> Optional[T]:
        return await self.session.get(self.model, id)

    async def get_all(self, limit=100, offset=0) -> List[T]:
        q = select(self.model).limit(limit).offset(offset)
        result = await self.session.execute(q)
        return result.scalars().all()

    async def save(self, obj: T) -> T:
        self.session.add(obj)
        await self.session.flush() # get generated ID
        await self.session.refresh(obj)
        return obj

    async def delete(self, id: int) -> bool:
        obj = await self.get_by_id(id)
        if not obj: return False
        await self.session.delete(obj); return True

class OrderRepository(BaseRepository[Order]):
    def __init__(self, session): super().__init__(session, Order)
    async def get_by_user(self, user_id) -> List[Order]:
        q = select(Order).where(Order.user_id == user_id)
        return (await self.session.execute(q)).scalars().all()
```

A3. How do you build custom middleware in FastAPI?

```
import time, uuid
from starlette.middleware.base import BaseHTTPMiddleware

class ObservabilityMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        trace_id = request.headers.get("X-Trace-Id", uuid.uuid4().hex)
        request.state.trace_id = trace_id
        start = time.perf_counter()
        try:
            response = await call_next(request)
        except Exception as e:
            logger.exception(f"[{trace_id}] unhandled", exc_info=e)
            raise
        duration_ms = (time.perf_counter() - start) * 1000
        response.headers["X-Trace-Id"] = trace_id
        response.headers["X-Duration-Ms"] = f"{duration_ms:.1f}"
        metrics.histogram("http_duration_ms",
            duration_ms, tags=[f"path:{request.url.path}",
                              f"status:{response.status_code}"])
        return response

app.add_middleware(ObservabilityMiddleware)
app.add_middleware(CORSMiddleware, allow_origins=["https://app.co"])
```

A4. How do you implement a plugin / extension architecture in Python?

```
# Use entry points (setuptools) or a simple registry
from abc import ABC, abstractmethod

_PLUGINS = {}

def register(name):
    """Decorator to register a plugin by name."""
    def deco(cls):
        _PLUGINS[name] = cls; return cls
    return deco

class StorageBackend(ABC):
    @abstractmethod
    async def put(self, key, value): ...
    @abstractmethod
    async def get(self, key): ...

@register("redis")
class RedisBackend(StorageBackend):
    async def put(self, k, v): await redis.set(k, v)
    async def get(self, k): return await redis.get(k)

@register("couchbase")
class CouchbaseBackend(StorageBackend):
    async def put(self, k, v): coll.upsert(k, v)
    async def get(self, k): return coll.get(k).content_as[dict]

def get_backend(name): return _PLUGINS[name]()
```

A5. What is dependency injection in Python frameworks and why does it matter?

DI decouples construction from use — objects receive their dependencies instead of creating them. In FastAPI, `Depends()` is the DI mechanism. In tests, swap the real dependency for a mock using `app.dependency_overrides`. This makes services independently testable without a real database or Kafka broker.

```
# In production
async def get_order_svc(db=Depends(get_db)):
    return OrderService(OrderRepository(db))

# In tests — override without touching source code
app.dependency_overrides[get_order_svc] = lambda: FakeOrderService()
```

A6. How do you handle configuration management across environments?

```
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    environment: str = "dev"
    db_url: str
    redis_url: str
    kafka_brokers: str
    jwt_secret: str
    log_level: str = "INFO"
    max_workers: int = 4

    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"
        secrets_dir = "/run/secrets" # Docker/K8s secrets mount

@lru_cache
def get_settings() -> Settings:
    return Settings() # env vars override .env which overrides defaults

# Priority: CLI env vars > docker secrets > .env file > defaults
```

Never hardcode secrets. In production, use AWS Secrets Manager + boto3, or mount Kubernetes Secrets as env vars. Rotate secrets without redeployment.

A7. How do you build a rate limiter as a FastAPI dependency?

```
import time
from collections import defaultdict
from fastapi import Request, HTTPException

# Simple in-process sliding window (use Redis in multi-instance deployments)
_windows: dict = defaultdict(lambda: [])

def rate_limit(max_calls: int, window_secs: int = 60):
    def dependency(request: Request):
        key = request.client.host
        now = time.time()
        _windows[key] = [t for t in _windows[key] if now - t < window_secs]
        if len(_windows[key]) >= max_calls:
            raise HTTPException(429, "Rate limit exceeded",
                                headers={"Retry-After": str(window_secs)})
        _windows[key].append(now)
    return dependency

@app.get("/api/data", dependencies=[Depends(rate_limit(100, 60))])
async def get_data(): ...
```

A8. Explain the difference between Django, Flask, and FastAPI for building microservices

Aspect	Flask	FastAPI	Django/DRF
Best microservice fit	Simple, small services	High-throughput async APIs	Complex domain with admin
Async first	No (WSGI)	Yes (ASGI)	Partial
Auto OpenAPI docs	No	Yes	Via DRF spectacular
Request validation	Manual / marshmallow	Pydantic (automatic)	DRF serializers
ORM bundled	No	No	Yes (Django ORM)
Setup overhead	Minimal	Minimal	High (many conventions)
Best for	Prototypes, small services	Production async APIs	Admin-heavy, monolith

A9. How do you implement API versioning without breaking existing clients?

```
from fastapi import FastAPI, APIRouter

app = FastAPI()
v1 = APIRouter(prefix="/v1")
v2 = APIRouter(prefix="/v2")

@v1.get("/orders/{id}")
async def get_order_v1(id: int):
    return OrderV1Schema.from_orm(await svc.get(id))

@v2.get("/orders/{id}")
async def get_order_v2(id: int):
    return OrderV2Schema.from_orm(await svc.get(id))

app.include_router(v1)
app.include_router(v2)
# /v1/orders/1 and /v2/orders/1 live side-by-side
# Deprecate v1 with a Deprecation: true header + sunset date
```

A10. How do you implement health checks and readiness probes for Kubernetes?

```
from fastapi import FastAPI
from sqlalchemy import text
import redis.asyncio as aioredis

@app.get("/health/live")    # Liveness: is the process alive?
async def liveness():
    return {"status": "alive"}

@app.get("/health/ready")  # Readiness: can we serve traffic?
async def readiness():
    checks = {}
    try:
        await db.execute(text("SELECT 1")); checks["db"] = "ok"
    except: checks["db"] = "fail"
    try:
        await redis_client.ping(); checks["redis"] = "ok"
    except: checks["redis"] = "fail"

    status = 200 if all(v=="ok" for v in checks.values()) else 503
    return JSONResponse(checks, status_code=status)
```

A11. How do you implement pagination — offset vs keyset?

```
# Offset pagination – simple but slow at large offsets
@app.get("/orders")
async def list_orders(page: int = 1, size: int = 20, db=Depends(get_db)):
    q = select(Order).offset((page-1)*size).limit(size)
    return (await db.execute(q)).scalars().all()

# Keyset (cursor) pagination – O(log n), stable under inserts
@app.get("/orders/cursor")
async def list_cursor(after_id: int = 0, size: int = 20, db=Depends(get_db)):
    q = select(Order).where(Order.id > after_id).order_by(Order.id).limit(size)
    rows = (await db.execute(q)).scalars().all()
    next_cursor = rows[-1].id if len(rows) == size else None
    return {"data": rows, "next_cursor": next_cursor}
```

Always use keyset pagination for large datasets. Offset scanning row-by-row to skip is $O(n)$; keyset uses the index directly at $O(\log n)$.

A12. How do you implement background tasks and job queues in a Python framework?

```
# FastAPI BackgroundTasks – for lightweight fire-and-forget
@app.post("/orders")
async def create(order: OrderIn, bg: BackgroundTasks):
    saved = await svc.create(order)
    bg.add_task(send_confirmation_email, saved.id) # runs after response
    return saved

# Celery – for durable, retryable, distributed tasks
from celery import Celery
celery = Celery("app", broker="redis://localhost/0")

@celery.task(bind=True, max_retries=3, default_retry_delay=30)
def process_report(self, order_id):
    try:
        build_and_email_report(order_id)
    except Exception as e:
        raise self.retry(exc=e) # exponential back-off retry

# Trigger from API
process_report.delay(order_id)
```

A13. How do you handle file uploads securely in a Python API?

```
from fastapi import UploadFile, File, HTTPException
import boto3, magic, hashlib

ALLOWED_TYPES = {"image/jpeg", "image/png", "application/pdf"}
MAX_SIZE = 50 * 1024 * 1024  # 50 MB

@app.post("/upload")
async def upload(file: UploadFile = File(...)):
    content = await file.read()

    # Validate size
    if len(content) > MAX_SIZE:
        raise HTTPException(413, "File too large")

    # Validate MIME type by content (not extension – spoofable)
    mime = magic.from_buffer(content[:2048], mime=True)
    if mime not in ALLOWED_TYPES:
        raise HTTPException(415, f"Unsupported type: {mime}")

    # Generate a safe S3 key (never use the original filename)
    sha256 = hashlib.sha256(content).hexdigest()
    key = f"uploads/{sha256[:2]}/{sha256}"
    s3 = boto3.client("s3")
    s3.put_object(Bucket="my-bucket", Key=key, Body=content,
                  ContentType=mime, ServerSideEncryption="AES256")
    return {"key": key, "size": len(content)}
```

A14. What is the OpenAPI spec and how does FastAPI auto-generate it?

OpenAPI (formerly Swagger) is a standard JSON/YAML specification describing an API's endpoints, request/response shapes, auth, and examples. FastAPI introspects type hints and Pydantic models at startup to generate the spec, serving it at `/openapi.json` with interactive UI at `/docs` (Swagger) and `/redoc`. Customize with `response_model`, `responses`, `tags`, and `openapi_extra`.

```
@app.post("/orders",
          response_model=OrderOut,
          status_code=201,
          responses={409:{"description":"Duplicate order"}},
          tags=["Orders"],
          summary="Place a new order")
async def create_order(order: OrderIn): ...
```

A15. How do you implement circuit breakers in a Python service?

```
from circuitbreaker import circuit

@circuit(failure_threshold=5, recovery_timeout=30,
        expected_exception=requests.Timeout)
def call_payment_service(order_id, amount):
    return requests.post(f"{PAYMENT_URL}/charge",
                        json={"order":order_id,"amount":amount}, timeout=3)

# States: CLOSED (normal) → OPEN (failing, return fallback fast)
#         → HALF-OPEN (test recovery) → CLOSED

def charge_with_fallback(order_id, amount):
    try:
        return call_payment_service(order_id, amount)
    except CircuitBreakerError:
        # Circuit is open — payment svc is down
        publish_to_retry_queue(order_id, amount)
        return {"status":"queued", "message":"Payment will retry"}
```

Section B — Scaling Applications & Handling Challenges

B1. What is horizontal vs vertical scaling and which do you prefer for Python services?

	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Approach	Bigger machine (more CPU/RAM)	More instances behind a load balancer
Limit	Hardware ceiling	Theoretically unlimited
Cost	Exponential at high levels	Linear
Downtime	Usually requires restart	Rolling deploy — zero downtime
Complexity	Simple	Services must be stateless
Python preference	OK for DB tier	Preferred for app tier

For Python APIs: always design stateless — externalize sessions to Redis, files to S3, scheduled locks to DynamoDB. Then you can scale horizontally freely. Start vertical only for the database until you hit PostgreSQL limits, then add read replicas.

B2. How do you make a Python service stateless for horizontal scaling?

```
# WRONG – state in process memory (breaks with 2+ instances)
_sessions = {} # session lost if request hits different pod

# CORRECT – externalize all state
import redis.asyncio as aioredis, json

redis = aioredis.from_url("redis://redis-cluster:6379")

async def set_session(token, data, ttl_secs=1800):
    await redis.setex(f"sess:{token}", ttl_secs, json.dumps(data))

async def get_session(token):
    raw = await redis.get(f"sess:{token}")
    return json.loads(raw) if raw else None

# Any pod can serve any request – consistent session from Redis
```

B3. How do you scale a Python FastAPI service with Gunicorn + Uvicorn?

```
# gunicorn.conf.py
workers = 4 # 2–4 × CPU cores for I/O-bound
worker_class = "uvicorn.workers.UvicornWorker"
bind = "0.0.0.0:8000"
timeout = 30
keepalive = 5
max_requests = 1000 # restart worker after N requests (avoid leaks)
max_requests_jitter = 100 # avoid thundering herd restarts
accesslog = "-" # stdout for container logging

# Run: gunicorn -c gunicorn.conf.py src.main:app
# In K8s: set replicas=N in Deployment; HPA scales based on CPU/RPS
```

B4. What are the common bottlenecks when scaling a Python microservice to 20x traffic?

- **Database connections** — each pod opens its own pool. 10 pods × 20 connections = 200 connections, often exceeding DB limits. Fix: PgBouncer (connection pooler) in front of PostgreSQL; use async drivers with smaller pools.
- **Unindexed queries** — 1x traffic hides slow queries; 20x exposes them as full table scans. Fix: EXPLAIN ANALYZE, add indexes, use read replicas for reads.
- **Synchronous blocking calls** — one slow external API call blocks a thread/worker. Fix: async HTTP (httpx), circuit breakers, timeouts.
- **No caching** — repeated expensive DB reads. Fix: Redis cache, CDN for static.
- **Heavy serialization** — large Pydantic models on every response. Fix: projections, partial responses, response streaming.
- **Kafka consumer lag** — producers write 20x but consumers don't scale. Fix: increase partitions + consumer instances.

B5. How do you implement database connection pooling in Python?

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker

# Engine with tuned pool – created ONCE at startup, shared across requests
engine = create_async_engine(
    settings.db_url,
    pool_size=10,          # permanent connections per worker
    max_overflow=20,       # burst connections
    pool_timeout=30,       # wait up to 30s for a connection
    pool_recycle=1800,     # recycle connections every 30min
    pool_pre_ping=True,    # test connection before use (handles DB restarts)
    echo=settings.environment == "dev"
)

AsyncSessionLocal = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def get_db():
    async with AsyncSessionLocal() as session:
        yield session    # auto-closed, auto-rolled-back on exception
```

B6. Explain auto-scaling in AWS ECS / Kubernetes for a Python service

```
# Kubernetes HPA – scale Deployment based on CPU
# apiVersion: autoscaling/v2
# kind: HorizontalPodAutoscaler
# spec:
#   scaleTargetRef:
#     name: order-service
#   minReplicas: 2
#   maxReplicas: 20
#   metrics:
#     - type: Resource
#       resource:
#         name: cpu
#         target:
#           type: Utilization
#           averageUtilization: 70
#     - type: External    # custom metric – e.g. Kafka consumer lag
#       external:
#         metric:
#           name: kafka_consumer_lag
#         target:
#           type: AverageValue
#           averageValue: 1000
```

B7. How do you implement retry with exponential backoff in Python?

```
import asyncio, random
from typing import Callable, Type

async def retry_async(
    fn: Callable,
    *args,
    max_attempts: int = 4,
    base_delay: float = 1.0,
    max_delay: float = 60.0,
    exceptions: tuple = (Exception,),
    **kwargs
):
    for attempt in range(1, max_attempts+1):
        try:
            return await fn(*args, **kwargs)
        except exceptions as e:
            if attempt == max_attempts:
                logger.error(f"All {max_attempts} attempts failed"); raise
            delay = min(base_delay * (2 ** (attempt-1))
                        + random.uniform(0, 1), max_delay) # jitter
            logger.warning(f"Attempt {attempt} failed: {e}. Retry in {delay:.1f}s")
            await asyncio.sleep(delay)

# Usage
result = await retry_async(call_payment_api, order_id, exceptions=(httpx.TimeoutException,))
```

B8. What is the strangler fig pattern for zero-downtime migration?

Replace an old system incrementally by routing a percentage of traffic to the new service, increasing until the old one is retired. Use a proxy or API gateway to split traffic: e.g., 5% → new service, monitor errors, 20% → 50% → 100%. Python implementation: a simple reverse proxy or feature-flag-based router.

```
import random

def route_request(request, feature_flags):
    rollout_pct = feature_flags.get("new_order_service_pct", 0)
    if random.random() * 100 < rollout_pct:
        return forward_to("new-order-service", request)
    return forward_to("legacy-order-service", request)
```

B9. How do you scale a Kafka consumer group in Python?

```
# Kafka distributes partitions across consumers in a group
# Max parallelism = number of partitions

# Scale-out: run more consumer instances with the SAME group.id
# Kafka rebalances: with 6 partitions and 3 consumers → 2 partitions each
# If consumers > partitions → extra consumers are idle

consumer = Consumer({
    "bootstrap.servers": "kafka:9092",
    "group.id": "order-processor",      # ALL instances share this
    "partition.assignment.strategy": "cooperative-sticky", # smoother rebalance
    "max.poll.interval.ms": 300000,    # 5 min max processing time
})

# Monitor consumer lag – if lag grows, add instances or partitions
```

B10. What is load shedding and backpressure?

Load shedding: deliberately drop low-priority requests when the system is overloaded, rather than letting latency spike for everyone. Return 503 with a **Retry-After** header. **Backpressure:** signal upstream producers to slow down (bounded queues, flow control). In Python: bounded `asyncio.Queue(maxsize=N)` blocks the producer when the consumer is behind; in Kafka, producer throughput is naturally throttled if the broker is overwhelmed via `max.block.ms`.

```
import asyncio

queue = asyncio.Queue(maxsize=500) # backpressure: put() blocks if full

async def producer():
    async for event in event_stream():
        try:
            queue.put_nowait(event)
        except asyncio.QueueFull:
            metrics.increment("events_shed") # load shed
            # drop low-priority; DLQ or retry for high-priority
```

Section C — Kafka — Deep Dive Topics

C1. How does Kafka topic configuration affect performance and reliability?

Setting	Purpose	Recommendation
<code>num.partitions</code>	Parallelism for consumers	Start 6–12; change requires topic recreate
<code>replication.factor</code>	Fault tolerance (copies)	3 in production (survives 2 broker failures)
<code>min.insync.replicas</code>	Minimum replicas that must ack	2 (with RF=3); ensures durability
<code>retention.ms</code>	How long messages are kept	7 days for events; shorter for high-volume
<code>compression.type</code>	Compress batches	lz4 or snappy — good throughput/CPU trade
<code>cleanup.policy</code>	delete or compact	compact for changelog/state topics

C2. Explain Kafka consumer group rebalancing and the cooperative-sticky strategy

Rebalancing happens when a consumer joins or leaves a group, or a partition count changes. **Eager rebalancing** (default): all consumers stop, partitions are redistributed — causes a processing pause ("stop the world").

Cooperative-sticky rebalancing (Kafka 2.4+, `CooperativeStickyAssignor`): only the partitions that need to move are revoked and reassigned in two rounds — consumers not losing partitions keep processing. Set `partition.assignment.strategy=cooperative-sticky` in Python consumers.

C3. What is a Kafka Schema Registry and why use it?

A centralized repository for Avro/Protobuf/JSON schemas, ensuring producers and consumers agree on message structure. Prevents schema drift silently breaking consumers. Each message stores only a schema ID (4 bytes) + payload — much smaller than embedding the schema each time.

```
from confluent_kafka.avro import AvroProducer
from confluent_kafka import avro

ORDER_SCHEMA = avro.load("schemas/order.avsc")
producer = AvroProducer(
    {"bootstrap.servers": "kafka:9092",
     "schema.registry.url": "http://schema-registry:8081"},
    default_value_schema=ORDER_SCHEMA
)
producer.produce("orders", value={"order_id": "1", "total": 200.0})
# Consumer auto-fetches the schema from registry by ID in the message
```

C4. How do you implement exactly-once semantics in Kafka with Python?

```
from confluent_kafka import Producer, Consumer, KafkaError

# Step 1: Idempotent producer (deduplicates retried messages)
producer = Producer({
    "bootstrap.servers": "kafka:9092",
    "enable.idempotence": True,      # acks=all + retries auto-set
    "transactional.id": "order-svc-1" # unique per producer instance
})
producer.init_transactions()

# Step 2: Transactional produce + offset commit in one atomic operation
def process_with_eos(msg):
    producer.begin_transaction()
    try:
        result = transform(msg.value())
        producer.produce("processed-orders", value=result)
        producer.send_offsets_to_transaction(
            {TopicPartition(msg.topic(), msg.partition()): msg.offset()+1},
            consumer.consumer_group_metadata()
        )
        producer.commit_transaction()
    except:
        producer.abort_transaction(); raise
```

EOS adds overhead. Use it only when duplicate processing has real business consequences (payments, inventory). For most analytics/logging pipelines, at-least-once with idempotent consumers is the right trade-off.

C5. What are Kafka Streams and how does it compare to the Consumer API?

	Consumer API	Kafka Streams
Language	Python/Java	Java only (native)
State management	Manual (external DB)	Built-in RocksDB state stores
Windowing/aggregations	Manual	Built-in (tumbling, hopping, session)
Join streams	Complex to code	Built-in KStream-KTable joins
Python alternative	Consumer API + Redis for state	faust-streaming (Python Streams)

```
# Python Kafka Streams equivalent: faust
import faust

app = faust.App("order-processor", broker="kafka://kafka:9092")
orders = app.topic("orders")

# Aggregate daily revenue – windowed table
revenue_table = app.Table("revenue", default=float).tumbling(86400)

@app.agent(orders)
async def process(stream):
    async for order in stream:
        revenue_table[order.region] += order.total

if __name__ == "__main__":
    app.main()
```

C6. How do you monitor Kafka in production?

Key metrics to alert on:

- **Consumer lag per topic/partition** — the most critical. Growing lag = consumers can't keep up. Alert if lag > N messages or > N minutes behind. Expose via JMX → Prometheus + Grafana or Confluent Control Center.
- **Under-replicated partitions** — a partition without full replicas = data loss risk. Alert on >0.
- **Active controller count** — should always be exactly 1. 0 = cluster problem.
- **Request rate and latency** — produce/fetch latency spikes indicate broker pressure.

```
# Python consumer: report lag programmatically
def report_consumer_lag(consumer, partitions):
    committed = consumer.committed(partitions)
    high_watermarks = [consumer.get_watermark_offsets(p) for p in partitions]
    for i, part in enumerate(partitions):
        lag = high_watermarks[i][1] - (committed[i].offset or 0)
        metrics.gauge(f"kafka.consumer.lag", lag,
                      tags=[f"topic:{part.topic}", f"partition:{part.partition}"])
```

C7. How do you handle Kafka message ordering guarantees?

Kafka guarantees ordering only **within a partition**. All messages with the same key go to the same partition (via hash), preserving per-entity order. To maintain global order, use one partition (no parallelism). For per-customer order: use `customer_id` as the key.

```

producer.produce(
    topic="order-events",
    key=order.customer_id.encode(), # all customer's orders → same partition
    value=json.dumps(order).encode()
)
# Consumers see this customer's events in creation order – guaranteed

```

C8. Kafka security — authentication and authorization

```

# mTLS + SASL/SCRAM authentication
producer = Producer({
    "bootstrap.servers": "kafka:9093",
    "security.protocol": "SASL_SSL",
    "sasl.mechanism": "SCRAM-SHA-512",
    "sasl.username": settings.kafka_user,
    "sasl.password": settings.kafka_password,
    "ssl.ca.location": "/certs/ca.pem",
    "ssl.certificate.location": "/certs/client.pem",
    "ssl.key.location": "/certs/client.key",
})

# Kafka ACLs: grant only what's needed
# kafka-acls --add --allow-principal User:order-svc
#             --operation WRITE --topic orders
# kafka-acls --add --allow-principal User:billing-svc
#             --operation READ --topic orders --group billing

```

C9. How do you design a Kafka-based event sourcing system?

Event sourcing: the authoritative state is a log of events, not the current snapshot. Events are immutable, append-only facts. Rebuild state by replaying events. Kafka is a natural fit: topics are ordered, durable, and replayable.

```

# Every state change is an event
events = [
    {"event": "OrderPlaced", "order_id": "1", "total": 200},
    {"event": "PaymentTaken", "order_id": "1", "amount": 200},
    {"event": "OrderShipped", "order_id": "1", "tracking": "TRK1"},
]

# Rebuild current state from events
def rebuild_order(order_id):
    state = {}
    for msg in consumer_from_beginning("order-events", key=order_id):
        event = json.loads(msg.value())
        apply_event(state, event) # update state with each event
    return state

```

C10. How do you handle poison pill messages (bad messages that crash the consumer)?

```
import json

def safe_process(consumer, producer, msg):
    try:
        data = json.loads(msg.value())
        validate_schema(data)          # raises ValidationError on bad data
        process_order(data)
        consumer.commit(asynchronous=False)
    except json.JSONDecodeError as e:
        # Unparseable – cannot retry, go to DLQ immediately
        send_to_dlq(producer, msg, error=f"ParseError: {e}")
        consumer.commit(asynchronous=False)
    except ValidationError as e:
        send_to_dlq(producer, msg, error=f"ValidationError: {e}")
        consumer.commit(asynchronous=False)
    except TransientError:
        # Don't commit – retry this message on restart
        raise

def send_to_dlq(producer, original_msg, error):
    producer.produce("orders-dlq",
                     key=original_msg.key(),
                     value=original_msg.value(),
                     headers={"error":error.encode(), "original-topic":"orders".encode()})
    producer.flush()
```

Section D — Caching Mechanisms

D1. What are the main caching strategies? Explain each with trade-offs.

Strategy	How it works	Consistency	Use when
Cache-aside (Lazy)	App checks cache; miss → load DB → populate cache	Strong on fresh load	Read-heavy, infrequent writes
Write-through	Writes go to cache AND DB synchronously	Strong always	Read-heavy, can afford write latency
Write-behind (async)	Write to cache; async flush to DB	Eventual	High write throughput, tolerate lag
Read-through	Cache fetches from DB on miss (transparent)	Eventual after TTL	Library-managed caching
Refresh-ahead	Proactively refresh before TTL expires	Near-real-time	Predictable access patterns

D2. How do you implement cache-aside pattern with Redis in Python?

```
import redis.asyncio as aioredis
import json
from datetime import timedelta

redis_client = aioredis.from_url("redis://redis:6379", decode_responses=True)

async def get_order(order_id: int) -> dict:
    cache_key = f"order:{order_id}"

    # 1. Try cache first
    cached = await redis_client.get(cache_key)
    if cached:
        metrics.increment("cache.hit", tags=["entity:order"])
        return json.loads(cached)

    # 2. Cache miss – load from DB
    metrics.increment("cache.miss", tags=["entity:order"])
    order = await db.fetch_order(order_id)
    if order is None:
        raise NotFoundError(f"Order {order_id}")

    # 3. Populate cache with TTL
    await redis_client.setex(cache_key, timedelta(minutes=15), json.dumps(order))
    return order

async def update_order(order_id, data):
    order = await db.update_order(order_id, data)
    # Invalidate cache on write
    await redis_client.delete(f"order:{order_id}")
    return order
```

D3. How do you avoid cache stampede (thundering herd) on cache miss?

```
# Problem: 100 concurrent requests all miss the cache at the same time,
# all hit the DB simultaneously → DB overwhelmed

# Solution 1: SET NX (set if not exists) – only one request rebuilds
async def get_with_lock(key, fetch_fn, ttl=300):
    cached = await redis_client.get(key)
    if cached: return json.loads(cached)

    lock_key = f"lock:{key}"
    acquired = await redis_client.set(lock_key, "1", nx=True, ex=10)

    if acquired:
        try:
            value = await fetch_fn()
            await redis_client.setex(key, ttl, json.dumps(value))
            return value
        finally:
            await redis_client.delete(lock_key)
    else:
        # Another request is rebuilding; wait briefly and retry
        await asyncio.sleep(0.1)
        return await get_with_lock(key, fetch_fn, ttl)

# Solution 2: Serve stale while refreshing (stale-while-revalidate)
# Store: {"data": ..., "expires_at": timestamp, "soft_ttl": timestamp}
# Serve stale immediately; trigger async refresh if past soft_ttl
```

D4. What are Redis eviction policies and which to choose?

Policy	Evicts	Best for
noeviction	Returns error when full	Databases (never lose data)
allkeys-lru	Least recently used of all keys	General cache
volatile-lru	LRU of keys with TTL only	Mix of persistent + cached
allkeys-lfu	Least frequently used	Skewed access (some hot keys)
allkeys-random	Random key	Uniform access, care little which

For a pure cache: `allkeys-lru` or `allkeys-lfu` depending on your access pattern. Set `maxmemory-policy` `allkeys-lru` and `maxmemory 2gb` in `redis.conf`.

D5. How do you implement function-level caching with Python decorators?

```
from functools import lru_cache, cached_property
import hashlib, pickle

# In-process LRU cache (single instance only)
@lru_cache(maxsize=512)
def get_exchange_rate(from_ccy: str, to_ccy: str) -> float:
    return fetch_from_api(from_ccy, to_ccy)

# Distributed Redis-backed cache decorator
def cache(ttl_secs=300, prefix="fn"):
    def deco(func):
        import functools
        @functools.wraps(func)
        async def wrapper(*args, **kwargs):
            raw_key = f"{prefix}:{func.__name__}:{args}:{sorted(kwargs.items())}"
            cache_key = hashlib.md5(raw_key.encode()).hexdigest()
            hit = await redis_client.get(cache_key)
            if hit: return pickle.loads(hit)
            result = await func(*args, **kwargs)
            await redis_client.setex(cache_key, ttl_secs, pickle.dumps(result))
            return result
        return wrapper
    return deco

@cache(ttl_secs=600)
async def get_product_catalog(category: str): ...
```

D6. What is cache invalidation and what are the common patterns?

"Cache invalidation is one of the two hard problems in CS." Three patterns: (1) **TTL expiry** — simplest, accept eventual consistency. (2) **Event-driven invalidation** — on write, publish a `cache.invalidated` event; all subscribers delete that key. (3) **Write-through** — update cache at the same time as DB. For distributed caches, use a Redis Pub/Sub or Kafka event to propagate invalidations.

```
# Pattern: Event-driven cache invalidation via Redis Pub/Sub
async def invalidate_product(product_id):
    await redis_client.delete(f"product:{product_id}")
    # Broadcast invalidation to all app instances
    await redis_client.publish("cache:invalidate",
                               json.dumps({"key": f"product:{product_id}"}))

# Listener on every app instance
async def listen_invalidations():
    pubsub = redis_client.pubsub()
    await pubsub.subscribe("cache:invalidate")
    async for msg in pubsub.listen():
        if msg["type"] == "message":
            data = json.loads(msg["data"])
            local_cache.delete(data["key"]) # clear in-process L1 cache
```

D7. What is a two-level (L1/L2) caching architecture?

L1 = in-process memory (`dict`, `lru_cache`, or `cachetools TTLCache`) — microsecond access, limited to one instance. L2 = shared Redis — millisecond access, shared across all instances. On read: check L1 → L2 → DB. On write: invalidate both. L1 reduces Redis round-trips for extremely hot data.

```

from cachetools import TTLCache

l1 = TTLCache(maxsize=1000, ttl=30)  # 30 seconds in-process

async def get_config(key):
    if key in l1: return l1[key]      # L1 hit – no network
    val = await redis_client.get(key) # L2
    if val:
        l1[key] = json.loads(val); return l1[key]
    val = await db.fetch_config(key) # DB
    await redis_client.setex(key, 300, json.dumps(val))
    l1[key] = val; return val

```

D8. What is CDN caching and how do you control it from a Python API?

```

from fastapi.responses import JSONResponse

@app.get("/products/{id}")
async def get_product(id: int):
    product = await svc.get(id)
    response = JSONResponse(content=product)
    # Cache at CDN for 5 minutes; serve stale for 1 min while revalidating
    response.headers["Cache-Control"] = "public, max-age=300, stale-while-revalidate=60"
    response.headers["ETag"] = hashlib.md5(json.dumps(product).encode()).hexdigest()
    return response

# On product update: purge CDN cache via API (CloudFront example)
cf = boto3.client("cloudfront")
cf.create_invalidation(
    DistributionId=DIST_ID,
    InvalidationBatch={"Paths":{"Quantity":1,"Items":[f"/products/{id}"]},
                      "CallerReference":str(uuid4())})

```

Section E — Queue-Based Architectures

E1. What is the difference between a message queue and a message broker?

A **message queue** (SQS, RabbitMQ) is point-to-point: one producer, one consumer group. Messages are deleted after consumption. A **message broker** (Kafka, EventBridge) supports pub/sub: one producer, many independent consumers, with optional message retention for replay. Use queues for task offloading; use brokers for event streaming and fan-out.

E2. AWS SQS vs SNS vs Kafka — when to use which?

Service	Pattern	Retention	Fan-out	Best for
SQS Standard	Queue (P2P)	Up to 14 days	No	Task offloading, worker queues
SQS FIFO	Queue (ordered)	Up to 14 days	No	Ordered task processing, deduplication
SNS	Pub/sub (push)	None	Yes (to SQS, Lambda, HTTP)	Notifications, broadcast events
SNS + SQS (fan-out)	Pub/sub + queuing	Queue TTL	Yes	Fan-out to multiple processors
Kafka / MSK	Event streaming	Days to forever	Yes (consumer groups)	High-throughput streaming, replay, analytics
EventBridge	Event bus (rule-based)	None	Yes	AWS service integration, scheduled events

E3. How do you implement an SQS worker in Python?

```
import boto3, json, logging

sqs = boto3.client("sqs")
QUEUE_URL = "https://sqs.us-east-1.amazonaws.com/123/order-queue"

def process_messages():
    while True:
        resp = sqs.receive_message(
            QueueUrl=QUEUE_URL,
            MaxNumberOfMessages=10,      # batch of 10
            WaitTimeSeconds=20,          # long polling – cheaper
            VisibilityTimeout=60,        # hidden from other consumers for 60s
            AttributeNames=["ApproximateReceiveCount"]
        )
        for msg in resp.get("Messages", []):
            try:
                body = json.loads(msg["Body"])
                receive_count = int(msg["Attributes"]["ApproximateReceiveCount"])

                process_order(body)

                # Delete only after successful processing
                sqs.delete_message(QueueUrl=QUEUE_URL,
                                   ReceiptHandle=msg["ReceiptHandle"])
            except Exception as e:
                logging.error(f"Failed (attempt {receive_count}): {e}")
                # Leave in queue → visibility timeout expires → retry
                # After maxReceiveCount → moved to DLQ automatically
```

E4. What is the SNS + SQS fan-out pattern and why use it?

SNS delivers a message to multiple SQS queues in parallel. Each queue has its own consumer (service) processing the same event independently. Benefits: decoupling (adding a new consumer doesn't change the producer), resilience (one consumer being slow doesn't affect others), each queue has its own visibility timeout and DLQ.

```
# Terraform: SNS → SQS fan-out
# resource "aws_sns_topic" "order_events" { name = "order-events" }
# resource "aws_sqs_queue" "billing_q" { name = "billing-orders" }
# resource "aws_sqs_queue" "inventory_q" { name = "inventory-orders" }
# resource "aws_sns_topic_subscription" "billing" {
#   topic_arn = aws_sns_topic.order_events.arn
#   protocol = "sqs"
#   endpoint = aws_sqs_queue.billing_q.arn
# }

# Python: publish to SNS – SQS consumers receive independently
sns = boto3.client("sns")
sns.publish(
    TopicArn=ORDER_EVENTS_ARN,
    Message=json.dumps({"order_id": "1", "total": 200}),
    MessageAttributes={"event_type": {"DataType": "String", "StringValue": "OrderPlaced"}}
)
```

E5. How does a dead-letter queue (DLQ) work in production?

A DLQ is a secondary queue that receives messages that couldn't be processed after `maxReceiveCount` attempts. It captures "poison pills" without losing them. In production: monitor DLQ depth and alert on non-zero; run a periodic DLQ reprocessor after fixing the bug; send to PagerDuty/Slack on DLQ message arrival.

```
# Lambda triggered by DLQ – inspect and alert
import boto3, json

slack = boto3.client("lambda")

def handler(event, ctx):
    for record in event["Records"]:
        body = json.loads(record["body"])
        logger.critical(f"DLQ message: {body}")
        send_slack_alert(f":fire: DLQ message on order-queue: {body}")
        # Store in S3 for later analysis
        s3.put_object(Bucket="dlq-archive",
                     Key=f"orders/{ctx.aws_request_id}.json",
                     Body=json.dumps(body))
```

E6. How do you implement an outbox pattern to guarantee event publishing?

The problem: if you save to DB and then publish to Kafka/SQS, a crash between the two loses the event. The outbox pattern: write the event to an `outbox` table in the same DB transaction as your business data; a separate poller reads the outbox and publishes, then marks events as published. Atomicity of DB transaction = no lost events.

```
# 1. Save order + outbox event in ONE transaction
async with db.begin():
    order = Order(user_id=uid, total=200)
    db.add(order)
    db.add(OutboxEvent(
        aggregate_id=order.id,
        event_type="OrderPlaced",
        payload=json.dumps({"order_id":order.id,"total":200}),
        published=False
    ))

# 2. Outbox poller (runs every second)
async def outbox_relay():
    while True:
        events = await db.query(OutboxEvent).filter_by(published=False).limit(100).all()
        for evt in events:
            producer.produce("orders", key=str(evt.aggregate_id), value=evt.payload)
            producer.flush()
            evt.published = True
        await db.commit()
        await asyncio.sleep(1)
```

E7. What is CQRS (Command Query Responsibility Segregation)?

Separate the write model (Commands: create/update/delete) from the read model (Queries: optimized projections). Commands go through a service, publish events to Kafka. The read side consumes events and maintains a denormalized, read-optimized view. Allows independent scaling of reads vs writes and tailored query models (e.g., Elasticsearch for search, Redis for fast reads).

```
# Write side: command → service → event to Kafka
@app.post("/orders")
async def place_order(cmd: PlaceOrderCommand):
    order = Order.create(cmd)
    await order_repo.save(order)
    await kafka.publish("order.placed", OrderPlacedEvent.from_order(order))
    return {"id": order.id}

# Read side: Kafka consumer updates a Redis read model
@app.agent(kafka_topic("order.placed"))
async def update_read_model(events):
    async for ev in events:
        view = OrderSummaryView.from_event(ev)
        await redis.hset(f"order_summary:{ev.user_id}", ev.order_id, view.json())
```

Section F — Production Stability, Alerting & Monitoring

F1. What are SLI, SLO, and SLA? How do you implement them?

Term	Definition	Example
SLI (Indicator)	The metric you measure	% requests < 200ms
SLO (Objective)	Target value for the SLI	99.5% of requests < 200ms
SLA (Agreement)	Business contract — penalty if SLO missed	"99.5% uptime or 10% credit"
Error budget	1 - SLO; how much failure is allowed	0.5% → ~3.65 hours/month

The error budget is your deployment risk budget. If it's depleted, freeze feature deployments and focus on reliability. Healthy budget = can deploy freely. This creates engineering incentives aligned with reliability.

F2. How do you instrument a Python FastAPI service with Prometheus metrics?

```
from prometheus_client import Counter, Histogram, Gauge, generate_latest
from fastapi import FastAPI, Response
import time

REQUEST_COUNT = Counter("http_requests_total",
    "Total HTTP requests", ["method", "endpoint", "status"])
REQUEST_LATENCY = Histogram("http_request_duration_seconds",
    "Request latency", ["method", "endpoint"],
    buckets=[.005, .01, .025, .05, .1, .25, .5, 1, 2.5, 5])
IN_PROGRESS = Gauge("http_requests_in_progress", "Requests in flight")

@app.middleware("http")
async def metrics_middleware(request, call_next):
    IN_PROGRESS.inc()
    start = time.perf_counter()
    response = await call_next(request)
    duration = time.perf_counter() - start
    IN_PROGRESS.dec()
    endpoint = request.url.path
    REQUEST_COUNT.labels(request.method, endpoint, response.status_code).inc()
    REQUEST_LATENCY.labels(request.method, endpoint).observe(duration)
    return response

@app.get("/metrics")    # Scraped by Prometheus every 15s
def metrics(): return Response(generate_latest(), media_type="text/plain")
```

F3. How do you set up structured logging for production?

```
import logging, json, sys
from pythonjsonlogger import jsonlogger

class ContextLogger(jsonlogger.JsonFormatter):
    def add_fields(self, log_record, record, message_dict):
        super().add_fields(log_record, record, message_dict)
        log_record["service"] = "order-service"
        log_record["environment"] = settings.environment
        log_record["version"] = settings.app_version

handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(ContextLogger("%(timestamp)s %(level)s %(name)s %(message)s"))
logging.root.setLevel(logging.INFO)
logging.root.addHandler(handler)

# Add trace ID to every log in a request
import contextvars
trace_id_var = contextvars.ContextVar("trace_id", default="-")

logger = logging.getLogger("order")
logger.info("Order placed", extra={
    "trace_id": trace_id_var.get(),
    "order_id": order.id,
    "user_id": user.id,
    "total": order.total,
})
```

JSON logs are essential for production. CloudWatch Logs Insights, Elasticsearch, and Datadog all parse JSON natively. Unstructured text logs require fragile regex parsing.

F4. How do you implement distributed tracing in a Python microservices system?

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.jaeger.thrift import JaegerExporter
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.sqlalchemy import SQLAlchemyInstrumentor

# Setup once at startup
provider = TracerProvider()
jaeger = JaegerExporter(agent_host_name="jaeger", agent_port=6831)
provider.add_span_processor(BatchSpanProcessor(jaeger))
trace.set_tracer_provider(provider)

FastAPIInstrumentor.instrument_app(app) # auto-trace HTTP
SQLAlchemyInstrumentor().instrument(engine=engine) # auto-trace SQL

# Manual span for a specific business operation
tracer = trace.get_tracer("order-service")
with tracer.start_as_current_span("apply_coupon") as span:
    span.set_attribute("coupon.code", coupon_code)
    discount = apply_coupon(order, coupon_code)
    span.set_attribute("discount.amount", discount)
```

F5. What are the key alerting rules for a Python API in production?

Alert	Condition	Severity	Action
High error rate	5xx rate > 1% for 2 min	P1	Page on-call
High latency	p99 > 2s for 5 min	P2	Slack alert
Low throughput	RPS < 10% of baseline	P1	Page on-call (possible outage)
Kafka consumer lag	Lag > 10,000 messages for 10 min	P2	Slack + auto-scale
DLQ non-empty	DLQ messages > 0	P2	Slack alert + ticket
CPU > 80%	Sustained 5 min	P3	Auto-scale trigger
Memory near limit	> 85% of limit	P2	Investigate leaks
Service down	Health check failing	P1	Page on-call immediately

F6. How do you implement graceful shutdown in a Python service?

```
import signal, asyncio

shutdown_event = asyncio.Event()

def handle_shutdown(*_):
    logger.info("Shutdown signal received")
    shutdown_event.set()

signal.signal(signal.SIGTERM, handle_shutdown)
signal.signal(signal.SIGINT, handle_shutdown)

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    await db_pool.connect()
    kafka_producer.init_transactions()
    logger.info("Service ready")
    yield
    # Graceful shutdown
    logger.info("Draining connections...")
    await asyncio.sleep(5) # let in-flight requests finish
    kafka_producer.flush(timeout=30) # flush pending Kafka messages
    await db_pool.disconnect()
    logger.info("Shutdown complete")
# K8s: terminationGracePeriodSeconds: 30 must exceed this
```

F7. How do you handle database migrations safely in production?

```
# Alembic – never run in main app process on startup
# WRONG: alembic upgrade head in Dockerfile CMD
# CORRECT: run as a separate K8s Job before the new deployment rolls out

# Safe migration checklist:
# 1. ADD column with default (nullable first, then backfill, then NOT NULL)
# 2. Never DROP column in the same deploy as the code removing it
# 3. Create index CONCURRENTLY (no table lock)
# 4. Rename via: add new col, dual-write, backfill, cut over, drop old

# Alembic migration example
def upgrade():
    op.add_column("orders", sa.Column("region", sa.String(10), nullable=True))
    op.execute("UPDATE orders SET region = 'UNKNOWN' WHERE region IS NULL")
    op.alter_column("orders", "region", nullable=False)
    op.create_index("ix_orders_region", "orders", ["region"],
                    postgresql_concurrently=True)
```

F8. What is a canary deployment and how do you monitor it?

Route a small percentage of traffic (5-10%) to the new version, monitor key metrics (error rate, p99 latency, business KPIs), and automatically roll back if thresholds are breached. In K8s: use Argo Rollouts or Flagger to automate canary analysis against Prometheus metrics.

```
# Argo Rollouts analysis template
# successCondition: result[0] < 0.01  (error rate under 1%)
# failureCondition: result[0] >= 0.05  (abort if >5%)
# metric: sum(rate(http_requests_total{status=~"5.."}[2m]))
#         / sum(rate(http_requests_total[2m]))

# Python: emit business metrics that canary analysis monitors
ORDER_FAILURES = Counter("orders_failed_total", "Orders that failed", ["reason"])
ORDER_REVENUE = Counter("orders_revenue_total_cents", "Revenue in cents")
```

F9. How do you implement feature flags in Python for safe releases?

```
from functools import lru_cache
import boto3, json

# Flags stored in AWS AppConfig or Parameter Store
ssm = boto3.client("ssm")

@lru_cache(maxsize=1)    # cache for 60 seconds to avoid SSM latency
def get_flags() -> dict:
    raw = ssm.get_parameter(Name="/app/feature-flags")["Parameter"]["Value"]
    return json.loads(raw)

def is_enabled(flag: str, user_id: str | None = None) -> bool:
    flags = get_flags()
    flag_config = flags.get(flag, {"enabled": False})
    if not flag_config["enabled"]: return False
    if pct := flag_config.get("rollout_pct"):
        return hash(user_id or "") % 100 < pct
    return True

if is_enabled("new-checkout-flow", user_id=user.id):
    return await new_checkout(order)
else:
    return await legacy_checkout(order)
```

Section G — AI-First Technologies & Solutions

G1. What is a RAG (Retrieval-Augmented Generation) architecture?

RAG grounds an LLM's responses in your private/real-time data by retrieving relevant documents from a vector database at query time and injecting them into the prompt. The LLM answers based on retrieved context, not just training data. Prevents hallucination on domain-specific questions and keeps knowledge current without expensive fine-tuning.

```
User query
↓ Embed query (text → vector)
↓ Similarity search in vector DB (Pinecone / pgvector / OpenSearch)
↓ Top-k relevant chunks returned
↓ Assemble prompt: context + query
↓ LLM (Bedrock Claude / GPT-4) generates answer grounded in context
↓ Response to user (with source citations)
```

G2. How do you implement a RAG pipeline in Python with AWS Bedrock?

```
import boto3, json
from langchain_aws import BedrockEmbeddings, ChatBedrock
from langchain_community.vectorstores import OpenSearchVectorSearch
from langchain.chains import RetrievalQA

bedrock = boto3.client("bedrock-runtime", region_name="us-east-1")

# Embed documents and store in OpenSearch
embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0", client=bedrock)

vectorstore = OpenSearchVectorSearch.from_documents(
    documents, embeddings,
    opensearch_url="https://opensearch:443",
    index_name="knowledge-base"
)

# LLM
llm = ChatBedrock(model_id="anthropic.claude-sonnet-4-6", client=bedrock)

# RAG chain
rag = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5}),
    return_source_documents=True
)

@app.post("/ask")
async def ask(question: str):
    result = rag.invoke({"query": question})
    return {"answer": result["result"],
            "sources": [d.metadata["source"] for d in result["source_documents"]]}
```

G3. What is LangChain and LangGraph? When do you use each?

LangChain: a framework for building LLM-powered applications — chains of prompts, retrievers, tools, and memory. Great for RAG pipelines, document Q&A, and tool-using chatbots. **LangGraph:** builds stateful, graph-based multi-agent workflows where nodes are LLMs or tools and edges represent conditional flows. Use LangGraph for complex multi-step reasoning, agent loops, and workflows requiring memory across steps.

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class AgentState(TypedDict):
    question: str; docs: list; answer: str

def retrieve(state): return {"docs": retriever.invoke(state["question"])}
def generate(state): return {"answer": llm.invoke(build_prompt(state))}
def grade(state): return "generate" if relevant(state["docs"]) else "retrieve"

graph = StateGraph(AgentState)
graph.add_node("retrieve", retrieve)
graph.add_node("generate", generate)
graph.add_conditional_edges("retrieve", grade, {"generate":END, "retrieve":"retrieve"})
graph.set_entry_point("retrieve")
app_chain = graph.compile()
```

G4. What are vector databases and how do you choose one?

Vector DB	Best for	Hosting	Notes
pgvector	Existing PostgreSQL stack	Self/RDS	SQL joins + vectors in one DB
OpenSearch (k-NN)	AWS-native, text + vectors	AWS managed	Good for hybrid search
Pinecone	High-scale production RAG	Managed SaaS	Best managed performance
Chroma	Local dev, prototyping	Self	Lightweight, easy setup
Weaviate	Multi-modal, graph + vectors	Self/Cloud	Built-in vectorizer
Faiss	In-process, huge scale	In-process	No persistence, Meta-built

G5. What is prompt engineering? Techniques for production systems.

```
def build_prompt(user_question: str, context_docs: list[str], user_role: str) -> str:
    context = "\n\n".join(context_docs[:3]) # top 3 docs
    return f"""You are a helpful assistant for {user_role} users.
    Answer ONLY based on the provided context. If the answer is not in the context,
    say "I don't have information about this."

Context:
{context}

Question: {user_question}

Instructions:
- Be concise and direct
- Cite the source section when relevant
- Never make up facts not in the context
"""

# Techniques:
# Zero-shot: just ask the question
# Few-shot: give 2-3 examples before the question
# Chain-of-thought: "Think step by step: ..."
# ReAct: Thought → Action → Observation loop for agents
```

G6. What is fine-tuning vs RAG? When to use which?

	RAG	Fine-tuning
Knowledge update	Real-time (update docs)	Requires retraining
Cost	Storage + retrieval cost	Training compute cost
Best for	Private/changing knowledge base	Style, tone, domain language patterns
Hallucination	Reduced (grounded in docs)	Not reduced
Use case	Company Q&A, document search	Domain-specific writing style, classification

In most enterprise AI use cases: start with RAG. Fine-tune only when you need the model to think/respond differently, not just to know new facts.

G7. How do you build an AI agent that uses tools (function calling)?

```
import boto3, json

bedrock = boto3.client("bedrock-runtime")

tools = [{
    "toolSpec": {
        "name": "get_order_status",
        "description": "Get the current status of an order by order ID",
        "inputSchema": {"json": {
            "type": "object",
            "properties": {"order_id": {"type": "string"}},
            "required": ["order_id"]
        }}
    }
}]

def run_agent(user_message: str):
    messages = [{"role": "user", "content": [{"text": user_message}]}]
    while True:
        resp = bedrock.converse(
            modelId="anthropic.claude-sonnet-4-6",
            messages=messages, toolConfig={"tools": tools})
        msg = resp["output"]["message"]
        messages.append(msg)
        if resp["stopReason"] == "tool_use":
            # Execute the tool
            tool_use = next(b for b in msg["content"] if "toolUse" in b)
            order_id = tool_use["toolUse"]["input"]["order_id"]
            status = get_order_status(order_id) # real function call
            messages.append({"role": "user", "content": [{"toolResult": {
                "toolUseId": tool_use["toolUse"]["toolUseId"],
                "content": [{"text": json.dumps(status)}]
            }
            }]}])
        else: # end_turn
            return msg["content"][0]["text"]
```

G8. What is AWS Bedrock and what does it offer?

AWS Bedrock is a fully managed service providing access to foundation models from Anthropic (Claude), Amazon (Titan), Meta (Llama), Mistral, Cohere, and others through a single API. Key features: no model management, pay-per-token, private VPC endpoints, AWS IAM integration, Bedrock Knowledge Bases (managed RAG), Bedrock Agents (managed agentic workflows), model evaluation, and Guardrails (content filtering, PII redaction, topic blocking).

G9. How do you implement streaming LLM responses in a FastAPI endpoint?

```
from fastapi.responses import StreamingResponse

@app.post("/chat/stream")
async def stream_chat(req: ChatRequest):
    async def token_generator():
        response = bedrock.invoke_model_with_response_stream(
            modelId="anthropic.claude-sonnet-4-6",
            body=json.dumps({
                "max_tokens": 1024,
                "messages": [{"role": "user", "content": req.message}]
            })
        )
        for event in response["body"]:
            chunk = json.loads(event["chunk"]["bytes"])
            if chunk.get("type") == "content_block_delta":
                text = chunk["delta"].get("text", "")
                yield f"data: {json.dumps({'token': text})}\n\n"
        yield "data: [DONE]\n\n"

    return StreamingResponse(token_generator(),
                           media_type="text/event-stream")
```

G10. How do you evaluate an LLM-powered system in production?

Don't rely on manual testing — build automated evaluation pipelines. Key metrics: faithfulness (does the answer match the context?), answer relevance (does it address the question?), context recall (was the right doc retrieved?). Use LLM-as-judge for qualitative metrics.

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_recall

# Build a golden test set of (question, ground_truth, context, answer)
results = evaluate(
    dataset=test_dataset,
    metrics=[faithfulness, answer_relevancy, context_recall]
)
print(results)
# faithfulness: 0.92  answer_relevancy: 0.88  context_recall: 0.95
# Run in CI - fail if faithfulness < 0.85 (hallucination risk)
```

G11. What is AI observability and how do you monitor LLM apps?

Track: token usage per request (cost), latency (time to first token, total), error rate, cache hit rate (semantic caching), user feedback (thumbs up/down), retrieval quality (context relevance), and guard-rail triggers. Tools: LangSmith, Arize Phoenix, MLflow, AWS CloudWatch + custom metrics.

```
import time
from prometheus_client import Histogram, Counter

LLM_LATENCY = Histogram("llm_response_seconds", "LLM latency", ["model"])
LLM_TOKENS = Counter("llm_tokens_used_total", "Tokens", ["model", "type"])
LLM_ERRORS = Counter("llm_errors_total", "LLM errors", ["model", "reason"])

async def tracked_llm_call(prompt, model="claude-sonnet-4-6"):
    start = time.perf_counter()
    try:
        response = await llm.ainvoke(prompt)
        LLM_TOKENS.labels(model, "input").inc(response.usage.input_tokens)
        LLM_TOKENS.labels(model, "output").inc(response.usage.output_tokens)
        return response
    except Exception as e:
        LLM_ERRORS.labels(model, type(e).__name__).inc(); raise
    finally:
        LLM_LATENCY.labels(model).observe(time.perf_counter() - start)
```

G12. What is semantic caching and how does it save LLM costs?

Cache LLM responses by semantic similarity of questions, not exact string match. "What is the return policy?" and "How do I return a product?" are semantically the same — cache one response for both. Libraries: GPTCache, LangChain's RedisSemanticCache.

```
from langchain.globals import set_llm_cache
from langchain_community.cache import RedisSemanticCache

set_llm_cache(RedisSemanticCache(
    redis_url="redis://redis:6379",
    embedding=BedrockEmbeddings(model_id="amazon.titan-embed-text-v2:0"),
    score_threshold=0.95 # 95% similarity = cache hit
))
# LLM calls now transparently check/populate the semantic cache
# Same question phrased differently → single LLM call → 90%+ cost savings
```

AI-First leadership talking point: "I approach AI features the same way I approach any production system — with observability, testing, and guardrails. I instrument token costs, latency, and error rates from day one. I build a golden test set before going to production and use automated evaluation to catch regressions in answer quality. And I treat LLM failures (hallucinations, refusals, slow responses) with the same circuit-breaker and fallback patterns I'd apply to any external service."